Quantum Advantage 보여주기

Classical Algorithm 과 Gate-based QAOA 비교

Homework #3

2026210344 이승원

0. 과제 배경

먼저 Quantum Supremacy 를 바로 증명하기보다, Quantum 방식이 실제로 어떤 장점을 가질 수 있는지 확인하는 Quantum Advantage 쪽에 집중했습니다. 이에 따라서 과제도 Binary Optimization 문제를 QUBO 형태로 만들고, 이를 gate-based quantum circuit 으로 직접 구성한 다음 classical 방식과 비교해보는 방향으로 이해하였습니다.

특히 수업에서 H, CNOT, RZ, RX gate 를 이용해 회로를 만들고, 처음에는 알 수 없는 angle parameter 를 classical algorithm 으로 찾아보려고 했습니다. 그리고 같은 문제를 classical 방식으로 풀었을 때와 비교해서 실행시간이나 memory 같은 계산 자원까지 확인해보는 것이 이번 과제(3)의 목적입니다. 수업 중 D-Wave 도 QUBO 를 푸는 Quantum Annealer 의 예로 다뤘지만, 이번 보고서에서는 직접 gate circuit 을 구성하는 QAOA 쪽에 집중했습니다.

1. 문제 선정 이유와 과제 수행 방향

소규모 Max-Cut 문제를 가져와봤으며, node 의 개수는 6 개입니다.

수업 내용에 맞춰 같은 QUBO 문제를 Classical 방식과 QAOA 로 각각 풀었을 때 어떤 차이가 있는지, 그리고 여기서 Quantum Advantage 라고 할 만한 부분이 있는지를 다시 확인하는 방향으로 정리하였습니다.

이 문제를 사용하면 가능한 해가 $2^6 = 64$ 개라서 Classical 방식으로 정확한 최적해를 직접 확인할 수 있고, QAOA 도 6-qubit 회로로 구성할 수 있습니다. 문제 자체가 너무 크지 않아서 QUBO 에서 Hamiltonian 과 gate circuit 으로 넘어가는 과정을 직접 이해하기에도 충분하다고 판단하였습니다.

2. Max-Cut 문제와 QUBO 표현

사용한 문제는 **6 개의 node 와 6 개의 edge 로 구성된 Max-Cut 문제**입니다. **각 node 를 0 또는 1 두 그룹 중 하나에 넣고, 서로 다른 그룹을 연결하는 edge 의 수를 최대화하면 되는 방식**입니다.

$$E = \{(0,1), (1,2), (2,3), (3,4), (4,0), (4,5)\}$$

각 node i 에 binary variable $x_i \in \{0,1\}$를 부여합니다. 하나의 edge (i,j)를 기준으로 두 node 가 서로 다른 그룹에 있으면 1 점, 같은 그룹에 있으면 0 점이 되도록 하면 다음 식을 사용할 수 있습니다.

$$c_{ij} = x_i + x_j - 2x_i x_j$$

Max-Cut 은 원래 점수를 크게 만드는 문제지만, QUBO 는 최소화 형태로 쓰기 위해 전체 Cut 에 음수를 붙여서 표현했습니다.

$$\text{Maximize } C(x) \;\Leftrightarrow\; \text{Minimize } Q(x) = -C(x)$$

이 그래프를 실제 QUBO 식으로 정리하면 다음과 같습니다.

$$Q(x) = -2(x_0+x_1+x_2+x_3) - 3x_4 - x_5 + 2(x_0x_1+x_1x_2+x_2x_3+x_3x_4+x_4x_0+x_4x_5)$$

가능한 비트열은 총 64 개이며, 전부 확인했을 때 **Max-Cut 의 정확한 최적값은 5***(Classical Algorithm 중 Brute Force 활용함)* 입니다. 따라서 QUBO 최소값은 −5 가 됩니다. 이 값은 이후 QAOA 결과가 어느 정도 좋은지 판단하는 기준으로 사용했습니다.

3. Classical 방식으로 먼저 풀기

Quantum 결과와 비교할 기준이 필요하기 때문에 먼저 Classical Brute Force 로 같은 문제를 풀었습니다. 방법은 000000 부터 111111 까지 가능한 64 개의 비트열을 모두 만들고, 각각의 QUBO 값을 계산한 뒤 가장 작은 값을 찾습니다.

이 방식은 문제 크기가 커지면 경우의 수가 빠르게 늘어난다는 단점이 있지만, 지금처럼 6 개의 binary variable 만 있는 문제에서는 최적의 해를 구할 때 효율적이라고 합니다, (아래는 Brute Force 구현 코드, 전체 실행코드를 따로 Python(.py) 파일 첨부 예정)

```
import itertools

def qubo(x):
    x0, x1, x2, x3, x4, x5 = x
    return (
        -2*(x0+x1+x2+x3) - 3*x4 - x5
        + 2*(x0*x1 + x1*x2 + x2*x3
             + x3*x4 + x4*x0 + x4*x5)
    )

best_value = float("inf")
```

```
best_solutions = []

for x in itertools.product([0, 1], repeat=6):
    value = qubo(x)
    if value < best_value:
        best_value = value
        best_solutions = [x]
    elif value == best_value:
        best_solutions.append(x)

print(best_value)
print(best_solutions)
```

결과는 Qmin = −5, 즉 Cutmax = 5 가 나왔습니다. 가능한 64 개 해 중에서 최적 Cut 을 만드는 비트열은 10 개였습니다. 이 값을 QAOA 에서 최적해가 얼마나 자주 나오는지 확인할 때 기준으로 사용하겠습니다.

4. 같은 문제를 QAOA 로 풀기

4.1 QUBO 를 Cost Hamiltonian 으로 바꾸기

이제 동일한 문제를 QAOA 에서는 0 과 1 로 적은 QUBO 식을 그대로 gate 에 넣을 수 없기 때문에, binary variable 을 Pauli-Z operator 로 바꿔주겠습니다. 이때 다음 관계를 사용합니다.

$$x_i = (I - Z_i) / 2$$

Max-Cut 에서는 두 node 가 서로 다른 그룹이면 1 점이 되어야 하는 규칙이 있습니다. 따라서 Pauli-Z 를 이용하면 하나의 edge 점수를 다음처럼 쓸 수 있습니다.

$$c_{ij} = (I - Z_i Z_j) / 2$$

두 qubit 의 값이 같으면 $Z_i Z_j = +1$ 이라서 0 점이 되고, 값이 다르면 $Z_i Z_j = -1$ 이라서 1 점이 됩니다. 결국 Classical Max-Cut 에서 사용한 같으면 0, 다르면 1 이라는 규칙을 quantum circuit 에서도 똑같이 구현해줬습니다.

4.2 Quantum Circuit 구성

node 가 6 개이므로 qubit 도 6 개를 사용하겠습니다. 먼저 모든 qubit 에 H gate 를 적용해서 여러 비트열 후보가 동시에 존재하는 superposition 상태를 만듭니다. 그다음 각 edge 마다 Cost 정보를 넣고, 마지막에 Mixer 를 적용합니다.

$$H \rightarrow CX \rightarrow RZ(-\gamma) \rightarrow CX \rightarrow RX(2\beta) \rightarrow \text{Measurement}$$

여기서 CX–RZ–CX 는 하나의 edge 에 대한 Cost interaction 입니다. 첫 번째 CX 는 두 qubit 이 같은지 다른지에 대한 정보를 target qubit 쪽에 잠시 반영하고, RZ 는 그 상태에 따라 phase 를 바꿉니다. 두 번째 CX 는 앞에서 임시로 바꿨던 상태를 다시 되돌립니다. 이 구조를 6 개의 edge 에 각각 적용하겠습니다.

Cost Layer 가 각 해의 점수 정보를 phase 에 넣는 역할이라면, RX gate 로 구성된 Mixer Layer 는 각 상태의 amplitude 를 섞어서 실제 측정 확률을 바꾸는 역할을 수행합니다. 결국 좋은 Cut 을 만드는 비트열이 더 자주 나오도록 γ 와 β 를 조정하는 것이 QAOA 의 핵심 이론입니다.

4.3 γ 와 β 를 찾는 방법

p=1 QAOA 에서는 Cost angle γ 와 Mixer angle β 두 개를 정해야 합니다. 문제는 이 값을 처음부터 알 수 없다는 것입니다. 그래서 이번 문제에서는 Classical Optimizer 인 COBYLA 를 사용하겠습니다.

$$(\gamma, \beta) \rightarrow \text{Circuit 실행} \rightarrow \text{Mean Cut 계산} \rightarrow \text{COBYLA 가 angle 수정} \rightarrow \text{반복}$$

따라서 quantum circuit 이 현재 angle 에서 결과를 내면, classical optimizer 가 그 결과를 보고 다음 γ 와 β 를 다시 정합니다. 이 과정을 여러 번 반복하면서 평균 Cut 이 좋아지는 방향으로 angle 을 찾습니다. 그래서 QAOA 는 Quantum 과 Classical 이 함께 돌아가는 hybrid algorithm 이라고 합니다.

5. Homework 실험 결과 및 Classical-QAOA 비교

앞에서 정리한 내용을 바탕으로 같은 6-node Max-Cut 문제를 Classical Brute Force 와 QAOA 로 직접 실행해 비교했습니다. QAOA 는 p=1, COBYLA optimizer 를 사용했고, angle 을 찾는 과정에서는 1,024 shots/evaluation, 최종 결과 확인에는 8,192 shots 를 사용했습니다. 초기 angle 에 따라 결과가 달라질 수 있어 총 5 번 독립적으로 실행했습니다.

Classical Brute Force 에서는 가능한 64 개의 비트열을 모두 확인했고, QUBO 최소값은 -5, Max-Cut 의 최적값은 5 로 확인되었습니다. 최적값을 만드는 비트열은 총 10 개였습니다.

QAOA 의 경우 5 번의 독립 실행 모두에서 Cut=5 인 최적해가 적어도 한 번은 측정되었습니다. 다만 QAOA 는 하나의 최적해를 바로 반환하는 방식이 아니라 확률분포를 만들기 때문에, 전체 측정 결과의 평균

Cut 은 3.7635 였고 최적해가 한 번의 측정에서 나올 확률인 P(opt)은 평균 30.21%였습니다. 제가 비교한 항목은 해의 품질(평균 Cut 등), 램사용량, 실행시간 이렇게 세가지 입니다.

비교 항목	Classical Brute Force	QAOA
사용 문제	6-node Max-Cut QUBO	동일함
사용 방법	64 개 경우 전수탐색	Quantum Circuit + COBYLA
정답	5	5
평균 Cut 값	Exact solution = 5	3.7635
최적해	전수 탐색함	P(opt) = 30.21%
실행시간	0.000032664 s	0.2019 s
최대 Ram 사용량 (RSS 의 Peak 값)	0.125 MB	1.148 MB

QAOA 는 run 마다 성능 편차도 있었습니다. 5 번 실행에서 Mean Cut 은 3.3458~4.3624, P(opt)은 9.58~62.18% 범위로 나타났습니다. 이는 초기 angle 과 optimization 결과에 따라 QAOA 의 성능이 달라질 수 있다는 점을 보여줍니다.

실행시간을 보면 Classical Brute Force 는 평균 약 0.000033 초가 걸린 반면, QAOA 는 한 번의 optimization 에 평균 약 0.202 초가 걸렸습니다. 이번 문제는 변수의 수가 6 개밖에 되지 않아 Classical 방식이 매우 빠르게 exact solution 을 찾을 수 있었습니다.

Memory 의 경우 Classical 은 Peak RSS 증가량이 0.125 MB, QAOA 는 평균 1.148 MB 로 측정되었습니다. 다만 QAOA 의 첫 실행에서는 Qiskit AerSimulator 초기화 과정도 포함되어 다른 실행보다 memory 증가량이 크게 측정되었을 것입니다. 따라서 memory 값은 절대적인 우위 지표라기보다 이번 실행환경에서의 참고값으로 보는 것이 적절합니다.

6. Quantum Advantage 에 대한 판단

이번 실험에서는 Classical Brute Force 와 QAOA 모두 Cut=5 인 최적해를 확인할 수 있었습니다. 다만 두 방식이 최적해를 찾는 과정과 필요한 계산량에는 차이가 있었습니다.

Classical Brute Force 는 가능한 64 개의 해를 모두 직접 계산하기 때문에 이번처럼 작은 문제에서는 매우 빠르게 exact solution 을 얻을 수 있었습니다. 반면 QAOA 는 H gate 로 초기 상태를 만들고, Cost Layer 와 Mixer Layer 를 적용한 뒤 COBYLA 를 이용해 γ 와 β 를 반복적으로 찾아야 합니다. 또한 각 angle 에서 여러 번의 measurement 가 필요하기 때문에 전체 계산시간이 더 길었습니다.

실제 측정 결과에서도 Classical 의 실행시간은 약 0.000033 초였지만 QAOA 는 평균 약 0.202 초가 걸렸습니다. QAOA 의 평균 Cut 은 3.7635 였고, P(opt)은 평균 30.21%였습니다. 따라서 이번 실험에서는 실행시간과 해의 안정성 측면에서 Classical 방식보다 QAOA 가 우수하다고 보기는 어려웠습니다.

따라서 이번 6-node Max-Cut 문제에서는 Quantum Advantage 를 확인하지 못했습니다. 하지만 이것이 QAOA 자체가 의미 없다는 뜻은 아닙니다. Quantum Advantage 는 충분히 큰 문제, 강한 Classical baseline 과의 공정한 비교, 실제 Quantum Hardware 등 여러 조건이 필요하기 때문에 소규모 simulator 실험에서 확인하기는 쉽지 않다고합니다.

이번 과제에서는 작은 문제에서 Classical 방식이 왜 유리한지 확인했으며, QUBO 를 Hamiltonian 과 Quantum Circuit 으로 바꾼 뒤 Classical Optimizer 와 함께 해를 찾는 QAOA 의 과정을 구현하고 이에 대한 결과 지표들을 비교해봤다는 점에 의미가 있다고 생각했습니다.

7. 참고자료

1) Quantum Computing 수업자료 및 Homework #3 강의 메모.

2) Lee, S. & Lee, H. (2026). QAOA Performance Evaluation for Small-Scale Max-Cut Problems under Limited Measurement Resources. 대한산업공학회 추계학술대회 발표자료.

3) Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028.

4) Hadfield, S. et al. (2019). From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz. Algorithms, 12(2), 34.

<AI 활용>

OpenAI ChatGPT 를 활용하여 QUBO/QAOA 개념 정리, Python 코드 작성 및 디버깅, 보고서 문장 표현을 참고하였습니다.